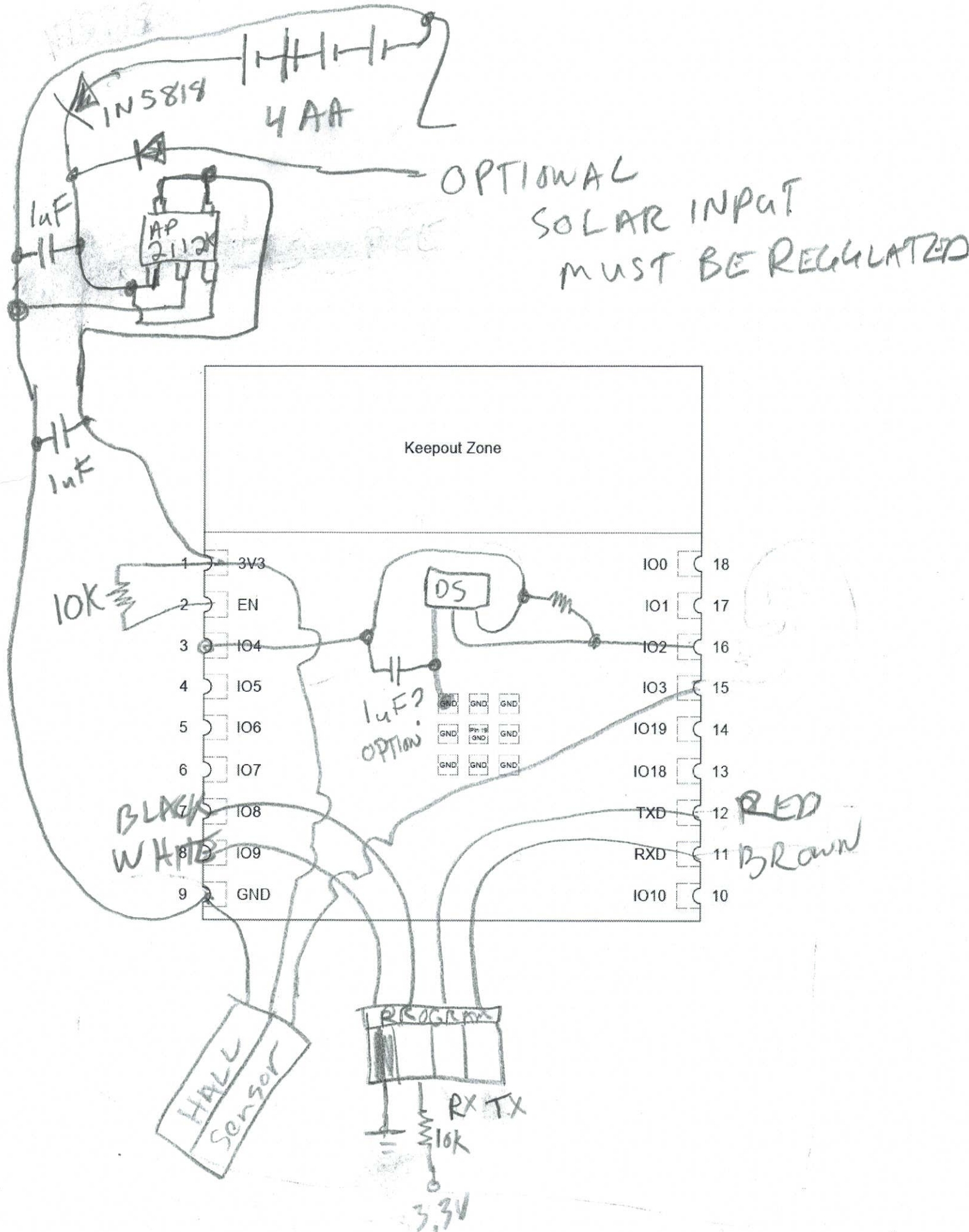
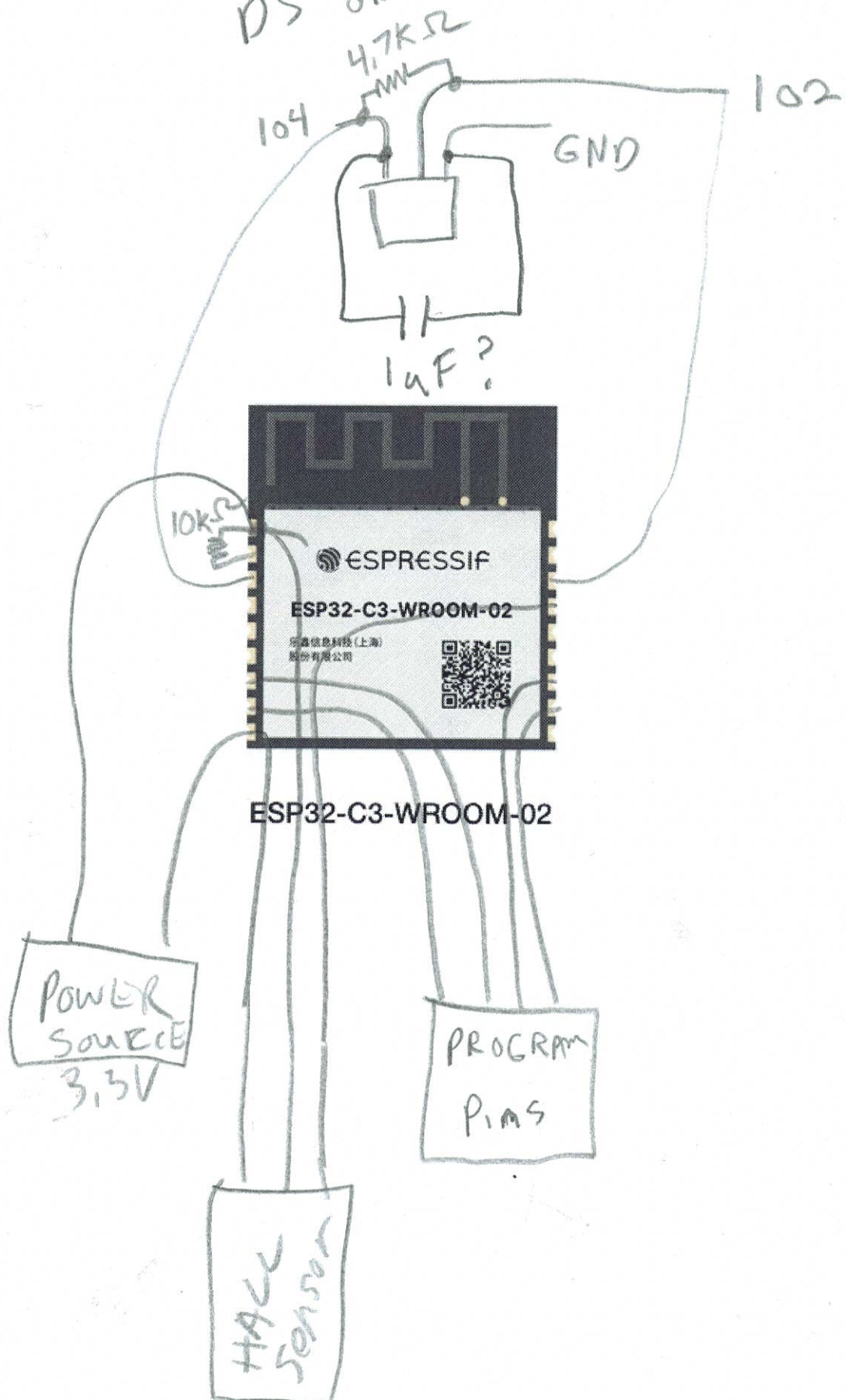




DS on the back



ESP32-C3-WROOM-02-N4 Technical Reference

1. MODULE PINOUT (ESP32-C3-WROOM-02-N4)

Orient the module with the PCB antenna at the top. Pins are numbered 1-18 counter-clockwise.

Pin	Label	Connection / Function
1	GND	Common Ground
2	3V3	Power Input (3.0V - 3.6V)
3	IO0	DS18B20 Data (DQ)
4	IO1	Hall Sensor Signal (Blue)
7	CHIP_EN	Enable (High) / Reset (Pull to GND)
8	IO9	Boot Mode: Pull GND to Program
11	IO2	Must be Pulled HIGH for Boot
13	RXD	Serial Receive (Connect to Bridge TX)
14	TXD	Serial Transmit (Connect to Bridge RX)
18	GND	Common Ground

2. PERIPHERAL WIRING

[DS18B20 (Flat Side)]	[55140 HALL SENSOR]
1: GND (Left)	RED: 3.3V
2: DQ (Middle)	BLACK: GND
3: VDD (Right)	BLUE: IO1 (Pin 4)

- **DS18B20:** Connect DQ (Pin 2) to Module Pin 3. Place a 4.7kΩ resistor between DQ and 3.3V.
- **XC6206 Regulator:** Pin 1 (GND), Pin 2 (VOUT/3.3V), Pin 3 (VIN/Battery).

3. PROGRAMMING VIA DEVKIT BRIDGE

To use an ESP32-C3 DevKit (SuperMini) as a USB-to-Serial bridge:

1. **Disable DevKit:** Connect DevKit **EN** to **GND**.
2. **Serial Links:** DevKit **TX** → Target **Pin 14**; DevKit **RX** → Target **Pin 13**.
3. **Power:** Share GND and 3.3V between boards.
4. **Flash Mode:** Hold Target **Pin 8 (IO9)** to GND while pulsing Target **Pin 7 (CHIP_EN)** to GND.

Integrated ESP32-C3-WROOM-02-N4 Wiring & Programming

1. MODULE PINOUT (ESP32-C3-WROOM-02-N4)

Pin	Label	Function / Connection
1	GND	Common Ground
2	3V3	Power Input (3.0V - 3.6V)
3	IO0	DS18B20 Data (DQ)
4	IO1	Hall Sensor Signal (Blue)
7	CHIP_EN	Enable (High) / Reset (Pull to GND)
8	IO9	Boot Mode: Pull GND to Program
11	IO2	Must be Pulled HIGH for Boot
13	RXD	Serial Receive (Connect to Bridge TX)
14	TXD	Serial Transmit (Connect to Bridge RX)
18	GND	Common Ground

2. VISUAL COMPONENT REFERENCE

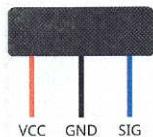
DS18B20 (TO-92)



XC6206 (SOT-23)



55140 Hall Sensor



Module Orientation



ESP32-C3 IoT Sensor Project

Component List & Wiring Specification

1. Core Component List

- **Microcontroller:** ESP32-C3 (SuperMini or equivalent)
- **Voltage Regulator:** XC6206P332MR (3.3V LDO, SOT-23)
- **Hall Effect Sensor:** 55140-3H-02-A (Magnetic Digital Switch)
- **Temperature Sensor:** DS18B20+ (OneWire, TO-92 package)
- **Resistors:**
 - 4.7k Ω Pull-up (for DS18B20+)
 - 10k Ω Pull-up (Optional for Hall Sensor GPIO 3)
- **Capacitors:**
 - 1 μ F Ceramic (Regulator Input Filter)
 - 1 μ F Ceramic (Regulator Output Stability)

2. Pin Connection Matrix

Component	Component Pin	ESP32-C3 Pin	Description
DS18B20+	Pin 1 (GND)	GND	Common Ground
DS18B20+	Pin 2 (DQ)	GPIO 2	Data line (Requires 4.7k Ω pull-up to 3.3V)
DS18B20+	Pin 3 (VDD)	3.3V	Sensor Power
Hall 55140-3H	Red Wire	3.3V	Sensor Power
	Black Wire	GND	Common Ground

Component	Component Pin	ESP32-C3 Pin	Description
Hall 55140-3H			
Hall 55140-3H	Blue/White	GPIO 3	Digital Signal (Wakes device from Deep Sleep)
XC6206 LDO	Pin 1 (GND)	GND	Regulator Ground Reference
XC6206 LDO	Pin 2 (Vout)	3.3V	Regulated Output to ESP32 & Sensors
XC6206 LDO	Pin 3 (Vin)	VBATT	Battery Input (3.7V - 6V)

Hardware Implementation Notes:

- Quiescent Current:** The XC6206P332MR is chosen for its $1.0\mu A$ draw, maximizing battery life during deep sleep.
- Deep Sleep Wakeup:** GPIO 3 is used for the Hall sensor because it supports RTC wakeup functionality.
- Filtering:** Ensure the $1\mu F$ capacitors are placed as close to the XC6206 pins as possible to prevent voltage ripple.

ESP32-C3 IoT Sensor Project

Component List & Wiring Specification

1. Core Component List

- **Microcontroller:** ESP32-C3 (SuperMini or equivalent)
- **Voltage Regulator:** XC6206P332MR (3.3V LDO, SOT-23)
- **Hall Effect Sensor:** 55140-3H-02-A (Magnetic Digital Switch)
- **Temperature Sensor:** DS18B20+ (OneWire, TO-92 package)
- **Resistors:**
 - 4.7kΩ Pull-up (for DS18B20+)
 - 10kΩ Pull-up (Optional for Hall Sensor GPIO 3)
- **Capacitors:**
 - 1μF Ceramic (Regulator Input Filter)
 - 1μF Ceramic (Regulator Output Stability)

2. Pin Connection Matrix

Component	Component Pin	ESP32-C3 Pin	Description
DS18B20+	Pin 1 (GND)	GND	Common Ground
DS18B20+	Pin 2 (DQ)	GPIO 2	Data line (Requires 4.7kΩ pull-up to 3.3V)
DS18B20+	Pin 3 (VDD)	3.3V	Sensor Power
Hall 55140-3H	Red Wire	3.3V	Sensor Power
	Black Wire	GND	Common Ground

Component	Component Pin	ESP32-C3 Pin	Description
Hall 55140-3H			
Hall 55140-3H	Blue/White	GPIO 3	Digital Signal (Wakes device from Deep Sleep)
XC6206 LDO	Pin 1 (GND)	GND	Regulator Ground Reference
XC6206 LDO	Pin 2 (Vout)	3.3V	Regulated Output to ESP32 & Sensors
XC6206 LDO	Pin 3 (Vin)	VBATT	Battery Input (3.7V - 6V)

Hardware Implementation Notes:

- Quiescent Current:** The XC6206P332MR is chosen for its $1.0\mu A$ draw, maximizing battery life during deep sleep.
- Deep Sleep Wakeup:** GPIO 3 is used for the Hall sensor because it supports RTC wakeup functionality.
- Filtering:** Ensure the $1\mu F$ capacitors are placed as close to the XC6206 pins as possible to prevent voltage ripple.

Source Component & Pin	Destination Component & Pin	Net Name / Signal	Description & Passive Components Required
Solar Panel (+) Anode	Diode 1 (1N5818) Anode	V_SOLAR	Solar supply input to OR-ing isolation circuit.
Battery (+) Anode	Diode 2 (1N5818) Anode	V_BATTERY	Battery supply input to OR-ing isolation circuit.
Diode 1 (1N5818) Cathode	AP2112K Pin 1 (VIN) AP2112K Pin 3 (EN)	V_REG_IN	Common tied cathode node supplying power to regulator input. Requires 1.0μF ceramic capacitor to GND.
Diode 2 (1N5818) Cathode	AP2112K Pin 1 (VIN) AP2112K Pin 3 (EN)	V_REG_IN	Common tied cathode node supplying power to regulator input. Requires 1.0μF ceramic capacitor to GND.
AP2112K Pin 5 (VOUT)	ESP32-C3 3V3 Pin	+3.3V	Regulated power rail. Requires 1.0μF ceramic capacitor to GND placed near regulator.
ESP32-C3 GPIO 4	DS18B20 VCC Pin	V_SENSOR_PWR	Switched peripheral power rail to eliminate sleep leakage. Requires 4.7kΩ pull-up resistor tied to GPIO 2.
ESP32-C3 GPIO 2	DS18B20 Data Pin	ONE_WIRE_BUS	Dallas 1-Wire communication line. Pulled up to V_SENSOR_PWR via 4.7kΩ resistor.
ESP32-C3 GPIO 3	Hall Effect Signal Out	HALL_SENSOR_PIN	External wakeup source. Utilizes internal pull-up configuration.
ESP32-C3 GPIO 8	Pull-up Resistor Terminal	STRAP_GPIO8	Strapping pin. Must be tied HIGH via 10kΩ resistor to +3.3V for normal and flash booting.
ESP32-C3 GPIO 9	DevKit Boot Jumper / Button	STRAP_GPIO9	Strapping pin. Grounded during reset to force serial bootloader flash mode.
ESP32-C3 CHIP_EN	DevKit RST Jumper / Button	CHIP_RESET	Hardware reset line. Requires 10kΩ pull-up resistor to +3.3V.
ESP32-C3 GPIO 20 (RX)	ESP32 DevKit TX Pin	UART_RXD	Serial data input line from bypassed USB-to-UART bridge.
ESP32-C3 GPIO 21 (TX)	ESP32 DevKit RX Pin	UART_TXD	Serial data output line to bypassed USB-to-UART bridge.
		GND	

Source Component & Pin	Destination Component & Pin	Net Name / Signal	Description & Passive Components Required
Common Ground Rail	AP2112K Pin 2 (GND) ESP32-C3 GND Hall Sensor GND DS18B20 GND DevKit GND Capacitor Grounds		Centralized star grounding point to reduce parasitic transient noise.

2. Program Functional Description

The program architecture is optimized for ultra-low power consumption by executing code purely within the `setup()` routine and terminating directly into an integrated deep sleep cycle. The firmware reads physical inputs, checks memory registers, logs historical datasets inside Real-Time Clock (RTC) static memory, manages peripheral power rails, and handles fast connection Wi-Fi handshakes.

2.1 Peripheral Utilization by Function

- System Initialization and Power Rail Control:** Immediately upon execution, the system configures the master system clock using `setCpuFrequencyMhz(80)`. Dropping the processing clock to 80MHz limits active phase dynamic current draw. The digital pin lock configurations are deactivated via `gpio_hold_dis()` for the hardware assets. GPIO 4 is asserted as an output and driven high to supply voltage to the DS18B20 sensor. A 200ms delay ensures stabilization of the thermal element's internal circuit topology before data conversion routines begin.
- Wakeup Cause Diagnostics:** The system invokes the `esp_sleep_get_wakeup_cause()` API to parse the reset vector. If the wake state evaluates to `ESP_SLEEP_WAKEUP_UNDEFINED`, the application classifies the cycle as a hard power-on event or manual hardware reset, assigning a trigger status code of 0. If evaluated as a `ESP_SLEEP_WAKEUP_GPIO`, the system queries the discrete logic state of GPIO 3. A low logic level denotes a magnetic closure event (trigger code 2), while a high logic level indicates a physical opening event (trigger code 3).
- 1-Wire Thermal Matrix Acquisition:** The DallasTemperature control layer initializes over the GPIO 2 wire bus. If valid sensor hardware addresses match, the asynchronous command `sensors.requestTemperatures()` forces the scratchpad configuration registers to update. The execution yields for 800ms to accommodate the full bitwise conversion processing interval. The value is processed using `sensors.getTempCByIndex(0)` and converted into a fixed-point integer scaling format inside the structure array.
- RTC Memory Buffering:** The system appends data packets directly into the `buffer` array residing inside the `RTC_DATA_ATTR` volatile memory section. This region retains structure parameters during deep sleep state transitions, removing the need to read and write non-volatile flash cells. This safeguards the internal flash memory and prevents structural cell degradation.

- **Network Subsystem Connection Handshake:** When the buffer matches threshold capacities or discrete trigger codes require real-time execution, `sendBufferedData()` commands the network hardware stack. `initWiFi()` configures a static IP layout utilizing local gate, mask, and DNS configurations. It bypasses multi-channel scanning intervals by passing exact values derived from `WIFI_CHANNEL` and `WIFI_BSSID` directly into the `WiFi.begin()` constructor, completing the Layer 2 authentication sequence within milliseconds.
- **Low-Power Deep Sleep Conditioning:** Once the payload transfer concludes or the connection times out, the firmware prepares for hardware isolation. The application isolates GPIO 4 by shifting it to a floating digital input mode and locking it with `gpio_hold_en()` to prevent internal leakage currents. Unused terminal traces (GPIOs 0, 1, 5, 6, 7, 10, 18, 19) are programmatically associated with low-impedance internal pull-down resistor arrays via `gpio_config()`. The radio hardware layer is safely de-energized using `WiFi.disconnect(true)`, `WiFi.mode(WIFI_OFF)`, and `btStop()` before starting the `esp_deep_sleep_start()` execution vector.

3. Firmware Source Code

The complete unadulterated software implementation configuration is appended below:

```
#include <WiFi.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include "config.h"
#include "driver/gpio.h"

#define HALL_SENSOR_PIN 3
#define ONE_WIRE_BUS 2
#define SENSOR_POWER_PIN 4
#define MAX_PACKETS 10

IPAddress local_IP(DEVICE_IP);
IPAddress gateway(DEVICE_GATEWAY);
IPAddress subnet(DEVICE_SUBNET);
IPAddress dns(DEVICE_DNS);

RTC_DATA_ATTR struct Packet {
    uint8_t id;
    uint8_t trigger;
    int16_t temp;
    int16_t rssi;
} buffer[MAX_PACKETS];

RTC_DATA_ATTR int packetCount = 0;
RTC_DATA_ATTR int16_t lastValidRSSI = 0;

OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

// Array of all valid GPIO pins on the ESP32-C3-WROOM-02 to configure before sleep
const gpio_num_t unusedPins[] = {
    GPIO_NUM_0, GPIO_NUM_1, GPIO_NUM_5, GPIO_NUM_6,
    GPIO_NUM_7, GPIO_NUM_10, GPIO_NUM_18, GPIO_NUM_19
};
```



```

const int numUnusedPins = sizeof(unusedPins) / sizeof(unusedPins[0]);

void setup() {
    setCpuFrequencyMhz(80);

    gpio_hold_dis((gpio_num_t)HALL_SENSOR_PIN);
    gpio_hold_dis((gpio_num_t)SENSOR_POWER_PIN);

    pinMode(SENSOR_POWER_PIN, OUTPUT);
    digitalWrite(SENSOR_POWER_PIN, HIGH);
    delay(200);

    pinMode(HALL_SENSOR_PIN, INPUT_PULLUP);

    int hallState = digitalRead(HALL_SENSOR_PIN);
    esp_sleep_wakeup_cause_t reason = esp_sleep_get_wakeup_cause();
    uint8_t currentTrigger = 1;

    if (reason == ESP_SLEEP_WAKEUP_UNDEFINED) {
        currentTrigger = 0;
    }
    else if (reason == ESP_SLEEP_WAKEUP_GPIO) {
        currentTrigger = (hallState == LOW) ? 2 : 3;
    }

    sensors.begin();
    float rawTemp = -127.0;
    if (sensors.getDeviceCount() > 0) {
        sensors.requestTemperatures();
        delay(800);
        rawTemp = sensors.getTempCByIndex(0);
    }

    if (packetCount < MAX_PACKETS) {
        buffer[packetCount].id = 0x01;
        buffer[packetCount].trigger = currentTrigger;
        buffer[packetCount].temp = (int16_t)(rawTemp * 100);
        buffer[packetCount].rssi = lastValidRSSI;
        packetCount++;
    }

    if (currentTrigger == 0 || currentTrigger == 2 || currentTrigger == 3 || packetCount >= 6) {
        sendBufferedData();
    }

    int wakeLevel = (hallState == LOW) ? 1 : 0;
    pinMode(HALL_SENSOR_PIN, INPUT_PULLUP);
    esp_deep_sleep_enable_gpio_wakeup(1ULL << HALL_SENSOR_PIN,
    (esp_deepsleep_gpio_wake_up_mode_t)wakeLevel);

    digitalWrite(SENSOR_POWER_PIN, LOW);
    pinMode(SENSOR_POWER_PIN, INPUT);
    gpio_hold_en((gpio_num_t)SENSOR_POWER_PIN);
    gpio_hold_en((gpio_num_t)HALL_SENSOR_PIN);

    WiFi.disconnect(true);
    WiFi.mode(WIFI_OFF);
    btStop();

    // Configure all unused pins with internal pull-downs to eliminate leakage current
    for (int i = 0; i < numUnusedPins; i++) {

```



```

    gpio_config_t io_conf = {};
    io_conf.intr_type = GPIO_INTR_DISABLE;
    io_conf.mode = GPIO_MODE_INPUT;
    io_conf.pin_bit_mask = (1ULL << unusedPins[i]);
    io_conf.pull_down_en = GPIO_PULLDOWN_ENABLE;
    io_conf.pull_up_en = GPIO_PULLUP_DISABLE;
    gpio_config(&io_conf);
}

esp_sleep_enable_timer_wakeup(600ULL * 1000000ULL);
esp_deep_sleep_start();
}

void initWiFi() {
    WiFi.config(local_IP, gateway, subnet, dns);
    // Connects instantly by bypassing the multi-channel scan
    WiFi.begin(WIFI_SSID, WIFI_PASS, WIFI_CHANNEL, WIFI_BSSID);
}

void sendBufferedData() {
    initWiFi();
    int wifiTimeout = 0;
    while (WiFi.status() != WL_CONNECTED && wifiTimeout < 20) {
        delay(500);
        wifiTimeout++;
    }

    if (WiFi.status() == WL_CONNECTED) {
        lastValidRSSI = (int16_t)WiFi.RSSI();

        WiFiClient client;
        if (client.connect(SERVER_IP, SERVER_PORT)) {
            client.setTimeout(2000);
            for (int i = 0; i < packetCount; i++) {
                if (i == packetCount - 1) {
                    buffer[i].rssi = lastValidRSSI;
                }

                char hexMsg[21];
                sprintf(hexMsg, "%02X%02X%04X%04X",
                    buffer[i].id, buffer[i].trigger,
                    (uint16_t)buffer[i].temp, (uint16_t)buffer[i].rssi);
                client.println(hexMsg);

                unsigned long start = millis();
                bool gotAck = false;
                while (millis() - start < 10000) {
                    if (client.available()) {
                        String response = client.readStringUntil('
');

                        if (response.indexOf("ACK") >= 0) {
                            gotAck = true;
                            break;
                        }
                    }
                    delay(50);
                }
                if (!gotAck) break;
            }
            client.stop();
        }
    }
}

```

```
    packetCount = 0;  
  }  
}  
  
void loop() {}
```


3. PROGRAMMING INSTRUCTIONS (DEVKIT BRIDGE)

1. **Bypass DevKit:** Connect DevKit **EN** pin to **GND** [cite: 7].
2. **Data Connections:** DevKit **TX** → Module **Pin 14**; DevKit **RX** → Module **Pin 13** [cite: 7].
3. **Flash Mode:** Hold Module **Pin 8 (IO9)** to **GND** while briefly connecting Module **Pin 7 (CHIP_EN)** to **GND** [cite: 7].
4. **Upload:** Start flashing from your IDE [cite: 7].